

Three Times Less Secure: A Human-Anchored Benchmark of LLM-Generated Infrastructure-as-Code

Animesh Shaw

GenIaC-SecBench Project

rana.animeshshaw73@gmail.com

Abstract—Large language models are increasingly used to author Infrastructure-as-Code (IaC), where a single insecure default is provisioned directly into a production cloud environment. Prior evaluations report absolute vulnerability counts for model-generated IaC, but without a human reference they cannot say whether models are actually *worse* than the engineers they assist. We present GenIaC-SecBench, a benchmark of 100 deployment scenarios stratified by architectural complexity, evaluated across 12 model configurations spanning four vendors and both open and closed weights, yielding 1,196 generated artifacts scanned by three independent policy engines (Checkov, Trivy, KICS) at complete coverage. Crucially, we scan a corpus of 634 human-authored IaC templates with the identical toolchain, providing the first size-matched human security baseline for this task. We find that vulnerability density is strongly inverse to artifact size (Spearman $\rho = -0.55$, $p < 10^{-77}$), so unmatched comparisons measure artifact size rather than security. Matched on declared-resource count, *every* model configuration falls in a narrow band of $3.21\times$ – $3.87\times$ the human vulnerability density, and the gap widens as tasks get simpler ($4.9\times$ at one resource, $1.4\times$ at twenty or more). We further decompose “reasoning” into three arms—standard generation, prompt-engineered chain-of-thought, and the vendor’s own extended-thinking API—and show that vendor extended thinking significantly outperforms prompted chain-of-thought (-12.0% , $p = 0.0013$), while prompted chain-of-thought alone is indistinguishable from standard generation (-1.3% , n.s.). Instrumenting token usage reveals why the effect is bounded: extended thinking consumes under 1% of the output budget on this task class. We also report two negative results: the intuitive hypothesis that more deployable models are more vulnerable is unsupported ($r = 0.158$, $p = 0.625$), and classical complete-case Friedman testing is *uncomputable* on realistic benchmark designs, motivating the Skillings–Mack statistic. All code, data, and regeneration scripts are released.

Index Terms—infrastructure as code, large language models, software security, empirical software engineering, static analysis, cloud security

I. INTRODUCTION

Infrastructure-as-Code (IaC) tools such as Terraform, AWS CloudFormation, Azure Resource Manager, and Kubernetes manifests declare cloud infrastructure as version-controlled artifacts. A misconfiguration in an IaC template is not a latent code defect: it is provisioned, and the resulting storage bucket, security group, or IAM role is exposed exactly as written. Security smells in IaC are both common and consequential [1], [2].

Large language models now generate substantial volumes of such code. A body of work has established that LLM-

generated *application* code contains vulnerabilities at non-trivial rates [3]–[5], and that developers using AI assistants may write less secure code while believing the opposite [6]. Evaluations specific to IaC are newer and thinner [7].

A recurring limitation unites these studies: they report absolute or normalized vulnerability counts *for models only*. Reporting that a model averages eight findings per declared resource invites the immediate question *compared to what?* Without a human reference measured through the same toolchain, such numbers cannot distinguish three very different worlds: models being genuinely careless, models simply emitting more infrastructure per file, or a scanner ruleset that flags any template heavily. This paper supplies the missing anchor.

Contributions.

- 1) **A size-matched human security baseline.** We scan 634 human-authored IaC templates with the same three engines and show that density must be compared within resource-count strata. Matched, every model configuration sits at $3.21\times$ – $3.87\times$ the human baseline (Section IV-C).
- 2) **A three-arm decomposition of “reasoning.”** Separating prompt-engineered chain-of-thought from vendor extended-thinking APIs shows the two are not interchangeable for security (Section IV-E).
- 3) **Direct measurement of reasoning engagement.** Extended thinking consumes $<1\%$ of output tokens on IaC, bounding the achievable effect (Section IV-E).
- 4) **A methodological correction for incomplete benchmark designs.** Complete-case Friedman retains zero usable blocks here; we adopt Skillings–Mack [8] (Section IV-D).
- 5) **Calibration of an LLM judge against three human experts,** delimiting where automated scenario review is trustworthy (Section IV-J).
- 6) **Two negative results,** reported as findings rather than omitted (Section IV-K).

II. BACKGROUND AND RELATED WORK

A. Security of LLM-generated code

Pearce et al. [3] found roughly 40% of Copilot completions in security-relevant contexts contained weaknesses. User stud-

ies show developers with AI assistance produce less secure code while reporting higher confidence [4], [6]. Large-scale comparisons across models confirm that security varies substantially by model [5]. Benchmarks such as SecurityEval [9] and correctness suites such as EvalPlus [10] target application code, where the unit of analysis is a function. IaC differs: the unit is a declarative resource graph, correctness is schema validity, and the security question is about defaults rather than control flow.

B. IaC security

Rahman et al. [1] catalogued seven recurring security smells in IaC scripts; subsequent work extended detection across languages [11] and studied practitioner behaviour [2], [12]. Policy engines including Checkov [13], Trivy [14], and KICS [15] operationalize such rules, commonly mapped to CIS Benchmarks [16]. The closest prior evaluation of *generated* IaC is Vargas et al. [7], which evaluates text-to-Terraform generation. Our study differs in scope—four IaC formats rather than one, complexity stratification, three independent engines, multi-rater human validation—and, most importantly, in providing a human security baseline rather than model-only counts.

C. Reasoning modes

Chain-of-thought prompting improves performance on reasoning-heavy tasks [17], [18], though recent evidence suggests gains concentrate in mathematical and symbolic domains [19]. Vendors now expose reasoning as a first-class API parameter with an explicit token budget [20]–[22]. These are distinct mechanisms: one changes the prompt, the other allocates computation. To our knowledge no prior security evaluation separates them—and, as we report in Section V, conflating them is easy to do accidentally.

D. LLM-as-a-judge

Model-based evaluation is widely used [23], [24] but exhibits known biases, including preference for self-generated content [25]. We therefore treat our judge as a secondary signal and calibrate it against three human experts.

III. METHODOLOGY

A. Scenarios

The benchmark comprises 100 natural-language deployment scenarios: 60 *simple* (single-service tasks) and 40 *complex* (multi-component architectures spanning networking, identity, data, and observability). Scenarios specify functional requirements only and never mention security controls, so that generated security posture reflects model defaults. Coverage spans AWS, Azure, GCP, and provider-agnostic Kubernetes across Terraform HCL, CloudFormation, ARM, and Kubernetes manifests.

TABLE I
MODEL CONFIGURATIONS. THE THREE CLAUDE OPUS 4.6 ROWS DIFFER ONLY IN REASONING MODE, ISOLATING THAT VARIABLE.

Configuration	Vendor	Reasoning mode
claude-opus-4-6	Anthropic	none
claude-opus-4-6-cot	Anthropic	prompted CoT
claude-opus-4-6-thinking	Anthropic	extended thinking (adaptive)
claude-sonnet-4-6	Anthropic	none
gpt-5	OpenAI	none
gpt-5-thinking	OpenAI	reasoning_effort=high
gpt-4o	OpenAI	none
gemini-3.1-pro	Google	none
gemini-3.7-flash	Google	none
llama3 / mistral / phi3	open	none (local)

B. Model configurations

Table I lists the 12 configurations. Beyond nine distinct models across four vendors, three configurations isolate reasoning mode on a fixed base model: *standard*, *-cot* (a chain-of-thought system-prompt suffix), and *-thinking* (the vendor extended-thinking API). All calls are stateless with a fixed system prompt requesting code-only output.

C. Validation and scanning

Generated artifacts are schema-validated (terraform validate, cfn-lint, ARM parsing, kubeconform), then scanned by three independent engines from three vendors—Checkov [13], Trivy [14], and KICS [15]—giving a convergent-validity argument no single ruleset provides. We report *complete* coverage: 1,196 of 1,196 artifacts scanned by all three engines, verified by a machine-readable coverage manifest regenerated on every run.

D. The human reference corpus

We draw 634 human-authored IaC templates from three public repositories of production-style and reference infrastructure. Files are filtered by structural heuristics (root keys such as `Resources:` or `apiVersion:`) to exclude non-IaC content. This corpus is scanned with the *identical* toolchain and configuration as the generated artifacts, so any ruleset bias applies equally to both sides of the comparison.

E. Metrics

Our primary metric is *vulnerability density*: total findings across all three engines divided by the number of declared resources, obtained by parsing each artifact’s syntax tree. Density normalizes for the fact that a template declaring fifty resources has more opportunity to be flagged than one declaring two. Resource counts are taken from the AST parse rather than from scanner output, which degrades silently when a file cannot be parsed (Section V).

F. Statistical procedure

Models are compared within complexity strata using the Skillings–Mack statistic [8], the generalization of the Friedman test [26] to incomplete block designs; Section IV-D explains

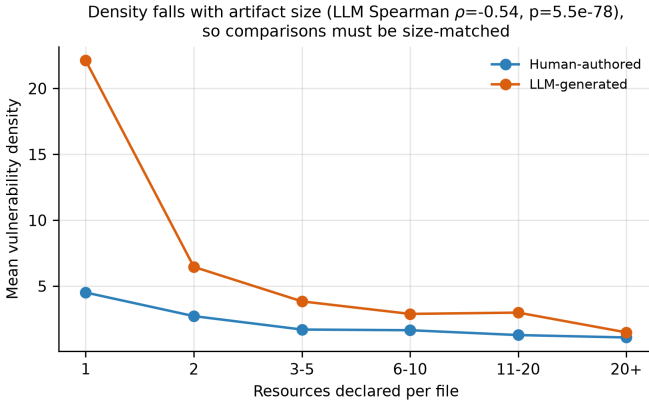


Fig. 1. Vulnerability density declines with artifact size in both corpora, requiring size-matched comparison.

why the classical test is unusable here. Post-hoc pairwise comparisons use Wilcoxon signed-rank tests [27] with Holm correction [28], following Demšar [29]. Counts are additionally modelled with a negative binomial GEE [30], [31] using $\log(\text{resource_count})$ as an exposure offset, so coefficients are per-resource incidence rate ratios. Negative binomial rather than Poisson is required empirically: the observed variance-to-mean ratio is 130.0. Human-model comparisons use Mann-Whitney U [32]; structural distributions are compared with two-sample Kolmogorov-Smirnov tests [33].

IV. RESULTS

A. Corpus

Generation yielded 1,196 of a possible 1,200 artifacts (99.7%). Four complex scenarios could not be generated within the model’s maximum output window even at 128k tokens—itsself a finding about the scale of architecture these prompts elicit. Scanning produced 38,803 findings (Checkov 14,017; KICS 14,033; Trivy 10,753).

B. Density must be size-matched

Vulnerability density falls steeply with artifact size (Fig. 1): in the human corpus it declines from 4.51 findings per resource in single-resource files to 1.10 in files declaring twenty or more, and across generated artifacts the association is strong and highly significant (Spearman $\rho = -0.55$, $p = 1.7 \times 10^{-78}$). Because the corpora differ in scale—human files average 5.31 declared resources, simple generations ≈ 3 , complex generations ≈ 50 —any unmatched comparison measures artifact size rather than security posture. All comparisons below are therefore computed within resource-count strata.

C. LLM-generated IaC is consistently less secure than human IaC

Fig. 2 compares generated and human artifacts within matched strata. The generated corpus exhibits significantly higher density in every stratum except the largest, with the ratio decreasing monotonically in artifact size: $4.9\times$ at one declared resource, $2.4\times$ at two, $2.3\times$ at three to five, $1.8\times$ at

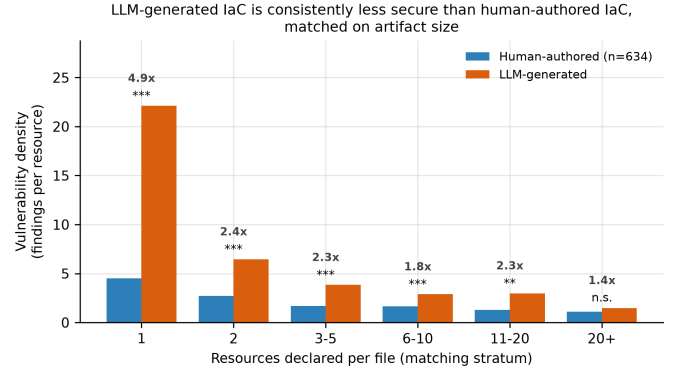


Fig. 2. Size-matched comparison against the human baseline. * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$ (Mann-Whitney U).

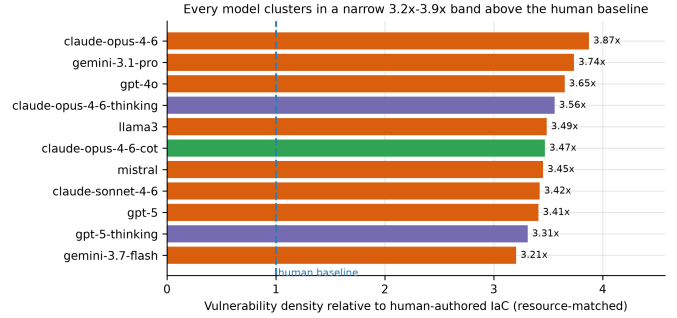


Fig. 3. Per-configuration density relative to the human baseline. Purple: vendor extended thinking. Green: prompted chain-of-thought.

six to ten, $2.3\times$ at eleven to twenty, and $1.4\times$ (not significant, $p = 0.058$) at twenty or more.

Aggregating per configuration over shared strata (Fig. 3) yields a strikingly narrow band: every configuration lies between $3.21\times$ and $3.87\times$ the human baseline. This consistency across four vendors, open and closed weights, and three reasoning modes suggests a property of current LLM-generated IaC rather than of any individual model.

Notably, the gap is *largest on the simplest tasks*. Where a human writes a minimal single-resource template, models emit substantially more flagged configuration—consistent with the structural over-generation reported in Section IV-I.

D. Models differ, but the classical test cannot show it

Skills–Mack rejects the null of equal model performance in both strata (simple: $\chi^2 = 69.3$, $df = 10$, $p = 6.0 \times 10^{-11}$, 60 blocks; complex: $\chi^2 = 81.2$, $df = 11$, $p = 8.7 \times 10^{-13}$, 40 blocks).

The classical alternative fails outright. Friedman requires complete blocks, and with 12 configurations and realistic coverage gaps *no scenario has every configuration present*: complete-case analysis retains **zero** blocks in both strata. This is not a matter of reduced power—the standard test in this literature [29] is simply uncomputable on a design of this shape. We regard this as a transferable methodological point: multi-model benchmarks routinely lose cells to refusals, quota

TABLE II
PAIRED REASONING-MODE CONTRASTS, SIMPLE STRATUM.

Contrast	n	Δ density	p
standard $\rightarrow$ extended thinking	60	-13.2%	0.012
prompted CoT $\rightarrow$ extended thinking	60	-12.0%	0.0013
standard $\rightarrow$ prompted CoT	60	-1.3%	0.238
gpt-5 $\rightarrow$ reasoning_effort=high	58	-10.2%	0.153

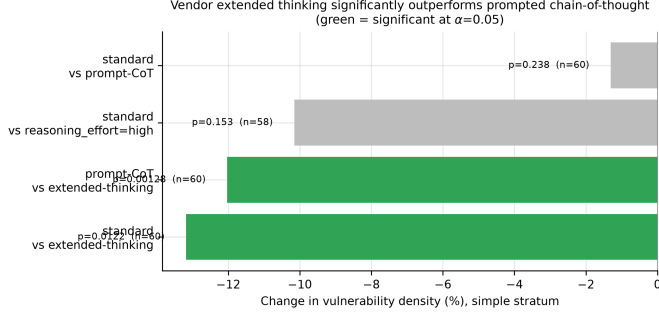


Fig. 4. Paired reasoning-mode contrasts on the simple stratum. Green bars are significant at $\alpha = 0.05$.

limits, and truncation, and should default to incomplete-block statistics.

E. Reasoning modes

Table II reports paired within-model contrasts on the simple stratum, where sample size is largest. Vendor extended thinking reduces density relative to standard generation (-13.2%, $p = 0.012$) and, more informatively, relative to prompted chain-of-thought (-12.0%, $p = 0.0013$). Prompted chain-of-thought alone is statistically indistinguishable from standard generation (-1.3%, $p = 0.238$).

Fig. 4 visualizes these contrasts. The practical implication is direct: instructing a model to “think step by step” does not confer the security benefit of paying for reasoning tokens. Given that prompted chain-of-thought is often treated as a free substitute, and that its benefits appear concentrated in mathematical and symbolic domains [19], this distinction matters for practitioner guidance.

Instrumenting the API explains why the effect, though real, is bounded. Fig. 5 shows reasoning-token expenditure: a median of 29 tokens on simple scenarios and 151 on complex ones, against median completion lengths of 886 and 18,533 tokens respectively—under 1% of the output budget on complex tasks, despite a generous configured allowance. The budget is a ceiling the model may underspend, not a target. A modest measured effect should therefore *not* be read as “reasoning does not help”; on this task class the mechanism is barely exercised.

F. Rate model

The negative binomial GEE with exposure offset yields incidence rate ratios from 0.73 to 4.64 against a `claude-opus-4-6` reference. The prompted-CoT arm

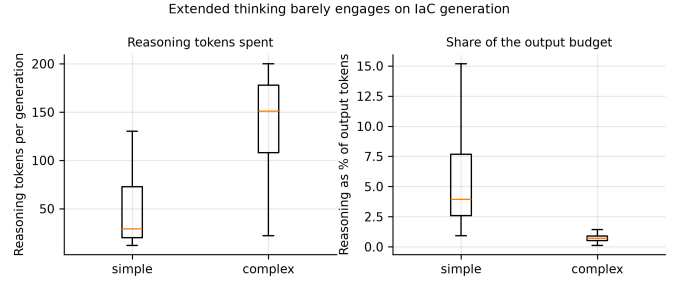


Fig. 5. Reasoning-token expenditure under extended thinking. The mechanism is barely engaged on IaC generation.

is significantly below the reference (IRR 0.73, 95% CI [0.54, 0.99], $p = 0.041$); `phi3` is highest (IRR 4.64, [3.12, 6.89], $p = 2.9 \times 10^{-14}$).

The `phi3` result inverts a naive reading. It records the fewest absolute findings and appears safest by raw count, but declares almost no parseable infrastructure; *per resource* it is the worst configuration measured. This is survivorship bias made explicit, and it is why absolute counts should not be used for cross-model comparison.

G. What models get wrong, and what the engines disagree about

Mapping findings to CIS Benchmark categories [16] (Fig. 6) shows Networking as the largest named category (4,554 findings), followed by Identity and Access Management (2,849), Logging/Monitoring (2,149), and Encryption (2,140). A large residual “Other” bucket (27,111) is dominated by Checkov rules that do not map cleanly onto CIS categories and is a tooling artifact rather than a vulnerability class. The prominence of networking and IAM is consistent with the security smells catalogued for hand-written IaC [1]: permissive ingress rules and over-broad role grants remain the dominant failure modes whether the author is human or machine.

Fig. 7 shows per-engine finding counts. The three engines disagree substantially in volume—Checkov and KICS report comparable totals while Trivy reports roughly 23% fewer—which is precisely the convergent-validity argument for using three independent vendors. A single-engine study would inherit that engine’s rule coverage as if it were ground truth. We note that our own earlier, partially-covered corpus would have supported different conclusions about inter-engine agreement, which is why we report coverage explicitly.

H. Schema validity

Schema-validity pass rates (Fig. 8) cluster between 27% and 35% for all frontier configurations and collapse for the small local models (`mistral` 8%, `llama3` 6%, `phi3` 5%). The absolute rates are low because complex scenarios demand multi-service architectures that must satisfy real provider schemas; they are nonetheless directly comparable across configurations, and they are what makes the survivorship correction in Section IV-K necessary.

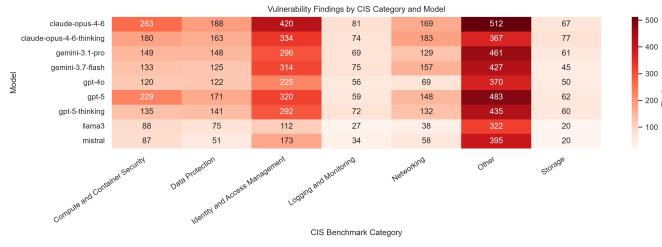


Fig. 6. Findings by CIS category and configuration. “Other” reflects rules without a clean CIS mapping and should not be read as a vulnerability class.

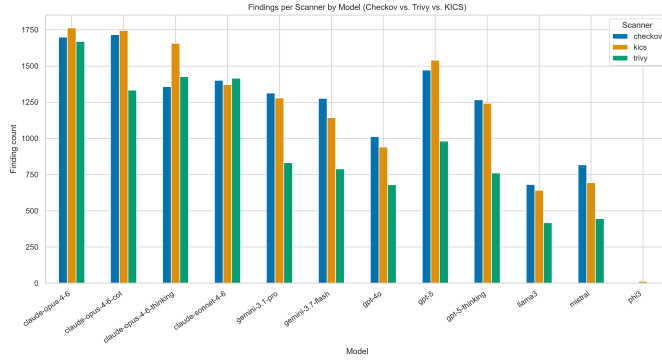


Fig. 7. Findings by engine. Volume disagreement across three independent vendors motivates multi-engine evaluation.

I. Structural divergence

Two-sample KS tests reject distributional equality with human-authored IaC for every configuration on every structural metric (36/36 tests, all $p < 0.05$). Generated templates declare 13–36 resources on average against 5.31 for humans. `gpt-4o` is closest ($D = 0.160$, $p = 0.021$); `phi3` is most divergent ($D = 0.990$). Models systematically over-generate infrastructure relative to human engineers.

J. Human evaluation and judge calibration

Three cloud-security practitioners independently scored a stratified sample of scenarios on four criteria. Inter-rater agreement (Fleiss’ κ [34]) was fair on real-world plausibility (0.391) and hallucination flagging (0.266), slight on architectural coherence (0.210), and negligible on security-test relevance (0.059). Under the Landis–Koch scale [35] the last is barely above chance: experienced engineers do not agree on whether a scenario poses a meaningful security decision. This is a substantive result about the difficulty of human security evaluation, not merely a limitation of our panel.

Against the human consensus, the LLM judge is substantial on hallucination detection (94.4% exact, Cohen’s $\kappa = 0.640$ [36]) and near-perfect on plausibility (quadratic-weighted $\kappa = 0.795$, 100% within one point), but near chance on architectural coherence ($\kappa = 0.177$, 27.8% exact). The boundary is actionable: automated judges are usable for factual verification, not for architectural assessment.

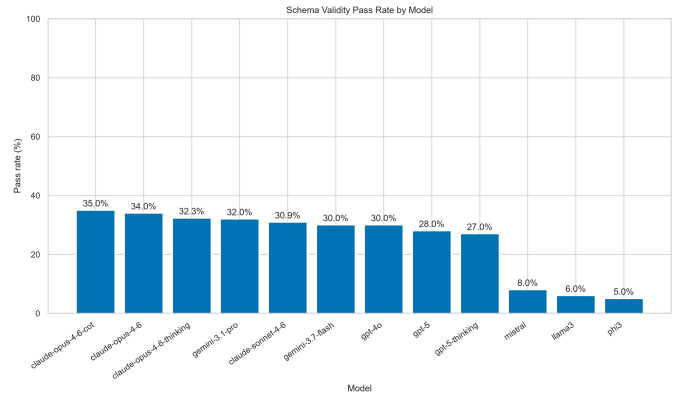


Fig. 8. Schema-validity pass rate by configuration.

K. Two negative results

First, the intuitive *validity–security* hypothesis—that models better at producing deployable code produce more vulnerable code—is unsupported. Across configurations, schema-validity pass rate and vulnerability density are uncorrelated ($r = 0.158$, $p = 0.625$; Spearman $\rho = 0.098$, $p = 0.761$). Pass rates cluster between 27% and 35% for all frontier configurations with no accompanying density relationship. Only the survivorship component holds: `phi3` passes 5% and produces almost no parseable infrastructure.

Second, as reported in Section IV-D, complete-case Friedman is uncomputable on this design.

V. THREATS TO VALIDITY

Construct. The human corpus is *not* a matched control: those templates were not written against our scenarios, and many are curated examples rather than production infrastructure. Example templates may be deliberately minimal, which could bias the baseline in either direction; we do not claim to know which. Vendor reasoning mechanisms also differ (token budget versus effort level) and are not pooled.

Internal. Scanner findings indicate policy deviations, not proven exploitability; “zero findings” means only that three rulesets flagged nothing. Checkov does not emit severity tiers without a commercial subscription, so severity analysis reflects Trivy and KICS only. Scenarios were authored by a model that also appears under test, an unquantified residual risk mitigated by requirements-only phrasing and independent human and judge review.

Measurement infrastructure. During this study we identified several defects that produced plausible-looking but incorrect results with no error raised: a platform encoding mismatch that aborted scanning for an entire configuration; a filename-handling rule that silently discarded one engine’s reports for configurations whose names contain a period; a schema mismatch that caused one engine’s findings to parse as zero for ten of twelve configurations; a resource-count source that degraded to a constant denominator, inflating an apparent effect to +2528% where the corrected value is −14%; and ungenerated scenarios being scored as zero-vulnerability. Each

would have survived review because the resulting numbers looked reasonable. We report this explicitly: in multi-tool benchmarks, derived counts must be reconciled against source artifacts rather than trusted, and we release coverage manifests and regeneration scripts so that any published figure can be recomputed.

External. Hosted models change without notice; results describe the configurations as accessed. Terraform dominates the corpus, and engine rule coverage differs by format. Statistical power for the paired reasoning contrasts is limited ($n \leq 60$), and only two model families expose reasoning arms.

VI. CONCLUSION AND FUTURE WORK

Measured against human-authored infrastructure through an identical toolchain and matched on artifact size, current language models produce IaC with approximately $3.2 \times - 3.9 \times$ the vulnerability density of human engineers, with remarkable consistency across vendors and weight availability. The gap is widest on the simplest tasks. Vendor reasoning modes deliver a real but modest improvement and meaningfully outperform prompted chain-of-thought, which alone provides no measurable security benefit—though the mechanism is barely engaged on this task class, consuming under 1% of output tokens.

Future work should pursue a matched human control authored against the same scenarios; larger samples and additional model families for the reasoning contrasts; a sensitivity sweep over reasoning-budget settings; exploitability triage to connect policy findings to realizable attack paths; deployment-time validation beyond static analysis; and improved rubric anchoring, given that expert agreement on security-test relevance was near chance.

DATA AVAILABILITY

The pipeline, statistical analysis code, figure generation, coverage manifests, and threats-to-validity documentation are released. Every figure and table is regenerated from the released data by a single command.

REFERENCES

- [1] A. Rahman, C. Parnin, and L. Williams, “The seven sins: Security smells in infrastructure as code scripts,” in *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.
- [2] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, “Exploring security practices in infrastructure as code: An empirical study,” *Empirical Software Engineering*, 2023.
- [3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions,” in *IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768.
- [4] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, “Lost at C: A user study on the security implications of large language model code assistants,” in *USENIX Security Symposium*, 2023, pp. 2205–2222.
- [5] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, “How secure is AI-generated code: A large-scale comparison of large language models,” *Empirical Software Engineering*, 2025.
- [6] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799.

- [7] G. Vargas, R. B. Mansilha, and D. Kreutz, “Security-first evaluation of text-to-Terraform generation,” *arXiv preprint arXiv:2608.02672*, 2026, accepted at SBSeg 2026.
- [8] J. H. Skillings and G. A. Mack, “On the use of a Friedman-type statistic in balanced and unbalanced block designs,” *Technometrics*, vol. 23, no. 2, pp. 171–177, 1981.
- [9] M. L. Siddiq and J. C. S. Santos, “SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques,” in *International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S)*, 2022, pp. 29–33.
- [10] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] N. Saavedra and J. F. Ferreira, “GLITCH: Automated polyglot security smell detection in infrastructure as code,” in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.
- [12] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “A systematic mapping study of infrastructure as code research,” *Information and Software Technology*, vol. 108, pp. 65–77, 2019.
- [13] Prisma Cloud / Bridgecrew, “Checkov: Policy-as-code for infrastructure as code,” <https://github.com/bridgecrewio/checkov>, 2026.
- [14] Aqua Security, “Trivy: Comprehensive security scanner,” <https://github.com/aquasecurity/trivy>, 2026.
- [15] Checkmarx, “KICS: Keeping infrastructure as code secure,” <https://github.com/Checkmarx/kics>, 2026.
- [16] Center for Internet Security, “CIS benchmarks,” <https://www.cisecurity.org/cis-benchmarks>, 2026.
- [17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [18] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [19] Z. Sprague, F. Yin, J. D. Rodriguez *et al.*, “To CoT or not to CoT? chain-of-thought helps mainly on math and symbolic reasoning,” *International Conference on Learning Representations (ICLR)*, 2025.
- [20] OpenAI, “Learning to reason with LLMs,” *OpenAI Technical Report*, 2024.
- [21] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [22] Anthropic, “Extended thinking,” <https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking>, 2026.
- [23] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [24] J. Gu, X. Jiang, Z. Shi *et al.*, “A survey on LLM-as-a-judge,” *arXiv preprint arXiv:2411.15594*, 2025.
- [25] A. Panickssery, S. R. Bowman, and S. Feng, “LLM evaluators recognize and favor their own generations,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [26] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [27] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [28] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [29] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [30] K.-Y. Liang and S. L. Zeger, “Longitudinal data analysis using generalized linear models,” *Biometrika*, vol. 73, no. 1, pp. 13–22, 1986.
- [31] J. M. Hilbe, *Negative Binomial Regression*, 2nd ed. Cambridge University Press, 2011.
- [32] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [33] F. J. Massey Jr., “The Kolmogorov-Smirnov test for goodness of fit,” *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.
- [34] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

- [35] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [36] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.